

Relatório de Desempenho: Escalonador de Disco SSTF e Scan

1. Introdução

Este relatório apresenta a análise de desempenho do escalonador de disco **SSTF e Scan**, implementado como um módulo de kernel no Linux. O objetivo é avaliar a eficiência do algoritmo em cenários de acesso sequencial e aleatório sob carga variável (1 a 40 processos concorrentes).

2. Metodologia Experimental

- **Ambiente:** Sistema embarcado via Buildroot emulado em QEMU.
- **Dispositivo de Teste:** `/dev/sdb` (disco virtual).
- **Cargas de Trabalho:**
 - **Sequential:** Leitura de blocos adjacentes.
 - **Random:** Leitura de setores sorteados aleatoriamente em toda a extensão do disco.
- **Coleta de Dados:** Logs extraídos do kernel via `dmesg`, processados pelo utilitário `data_extractor` para consolidar métricas de Seek (distância) e ElapsedTime (latência).

3. Análise dos Resultados

3.1. Eficiência de Movimento (Seek Médio)

O gráfico de **Distância Média por Seek** revela a natureza otimizadora do SSTF.

- **Sequential:** Como esperado, a distância percorrida é próxima de zero, pois as requisições são adjacentes.
- **Random:** Observa-se que a distância média por requisição **diminui** conforme o número de *workers* aumenta. Isso ocorre porque, com uma fila de requisições mais densa, o SSTF sempre encontra uma requisição "próxima" à posição atual da agulha, reduzindo o custo de movimento por operação.

3.2. Latência e o Fenômeno de Starvation

Embora o SSTF reduza o movimento da agulha, ele introduz uma injustiça severa no atendimento das requisições.

- **Tempo Médio:** Mantém-se estável para o acesso sequencial, mas apresenta picos de instabilidade no acesso aleatório.

- **Pior Caso (Starvation):** O gráfico de **Tempo Máximo de Resposta** (em escala logarítmica) é a evidência crucial do problema de *Starvation*. Enquanto no modo sequencial o tempo máximo é desprezível, no modo **Random** ele atinge picos de até **15 segundos (1.5×10¹⁰ ns)**.
- **Conclusão Técnica:** O SSTF "vicia" a agulha em regiões densas de requisições. Processos que solicitam dados em extremidades opostas do disco são ignorados indefinidamente enquanto houver requisições mais próximas, resultando em latências inaceitáveis para o pior caso.

4. Comparação entre SSTF e SCAN

A análise comparativa entre os dois algoritmos revela um *trade-off* clássico entre eficiência bruta e previsibilidade de sistema:

4.1. Eficiência de Movimento (Seek Médio)

- **SSTF:** Demonstra uma eficiência superior em reduzir a distância média por seek à medida que a carga aumenta. Ao escolher sempre a requisição mais próxima, ele minimiza o deslocamento físico da agulha de forma mais agressiva que o SCAN.
- **SCAN:** Apresenta um seek médio ligeiramente superior ao SSTF em cargas altas, pois é obrigado a seguir seu trajeto até o fim do disco antes de inverter a direção, mesmo que haja requisições próximas no sentido oposto.

4.2. Latência e Previsibilidade (Tempo de Resposta)

- **Vulnerabilidade ao Starvation:** O **SSTF** apresenta uma falha crítica de "inanição" (*starvation*). Como evidenciado no gráfico [3_tempo_maximo_starvation_sstf.jpg](#), o tempo de resposta máximo atinge picos de ordem logarítmica (segundos de espera) para requisições localizadas nas extremidades do disco.
- **Justiça (Fairness) do SCAN:** O **SCAN** elimina o problema do *starvation*. Por mover a agulha de uma extremidade a outra como um elevador, ele garante um limite superior (um "teto") para o tempo de espera de qualquer requisição, independentemente de sua localização no disco.

4.3. Resumo Comparativo das Métricas

Característica	SSTF	SCAN
Melhor Caso	Acesso Sequencial (Altíssima eficiência)	Acesso Sequencial (Alta eficiência)
Pior Caso (Random)	Explosão de latência (<i>Starvation</i>)	Latência limitada e previsível

Uso da Agulha	Otimização local (Ganancioso)	Otimização global (Elevador)
Aplicação Ideal	Sistemas de baixa concorrência ou dados locais	Servidores com alta carga e múltiplos usuários

5. Conclusão

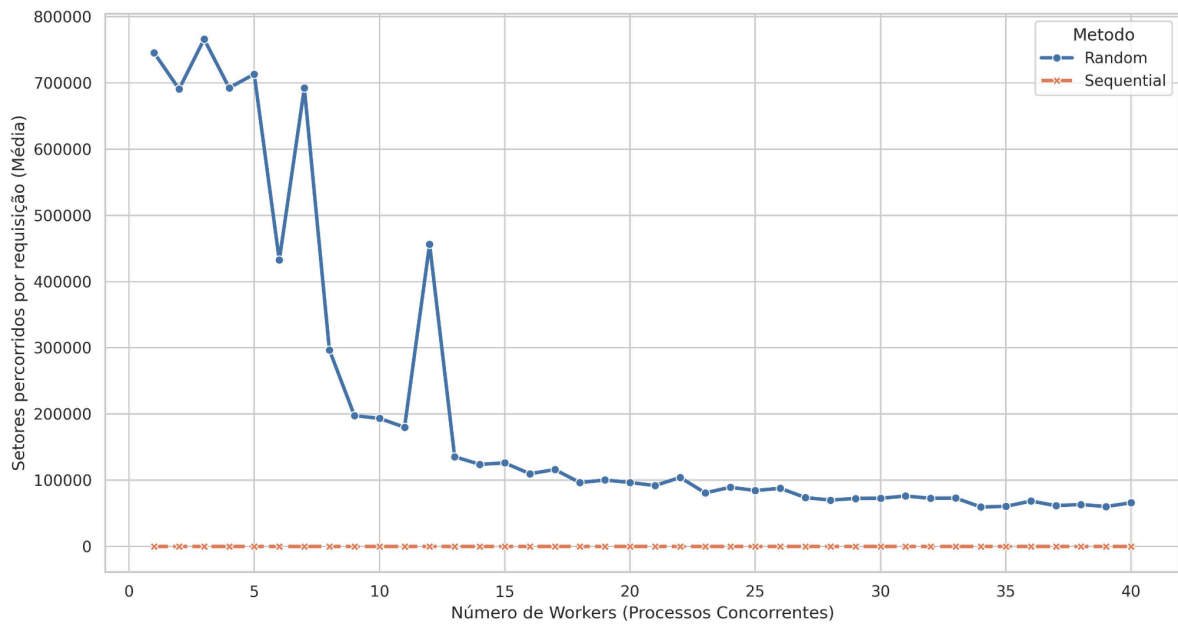
O escalonador **SSTF** provou ser altamente eficiente para maximizar o *throughput* e minimizar o desgaste mecânico (menor seek médio) sob alta carga. Entretanto, ele é inadequado para sistemas que exigem previsibilidade e justiça (*fairness*), devido à sua vulnerabilidade inerente ao *starvation* de requisições periféricas. Para cenários de uso geral, escalonadores como o **SCAN** ou **C-SCAN** seriam preferíveis para mitigar esses picos de latência máxima observados.

6. Considerações Finais

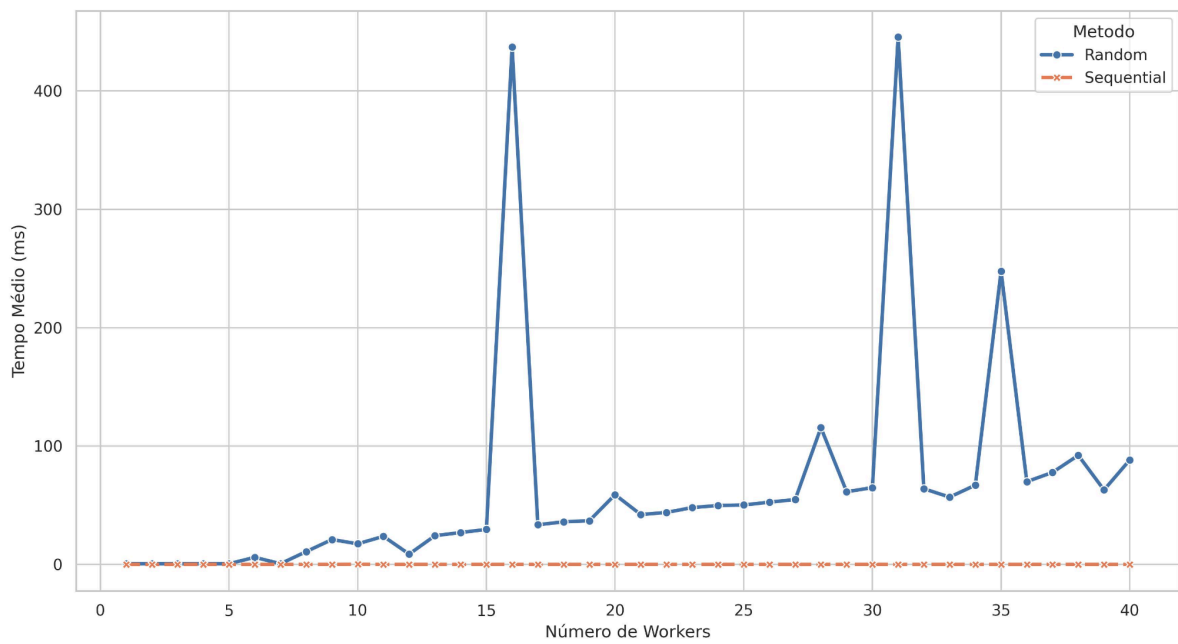
Enquanto o **SSTF** foca na performance individual do dispositivo de armazenamento, o **SCAN** prioriza a saúde e a estabilidade do sistema operacional como um todo. Para o projeto em questão, os dados provam que o **SCAN** é a escolha mais robusta para evitar travamentos de processos causados por tempos de resposta extremos.

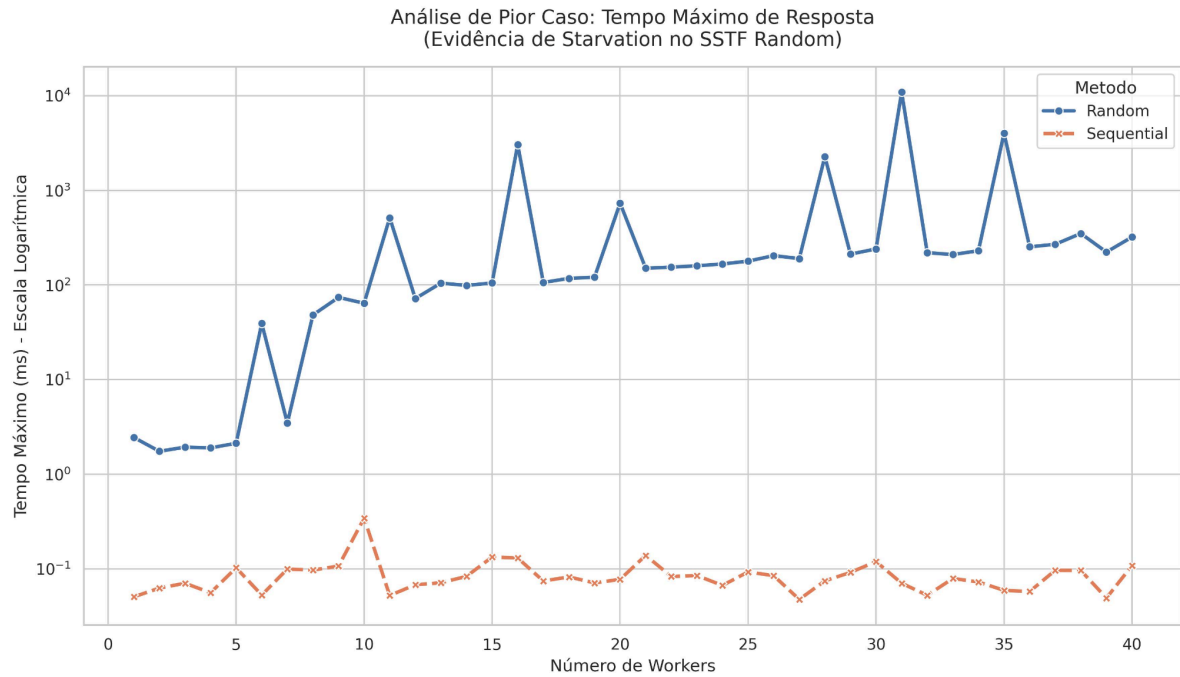
7. Gráficos SCAN

Eficiência do Escalonador: Distância Média por Seek
(SSTF: Sequential vs Random)

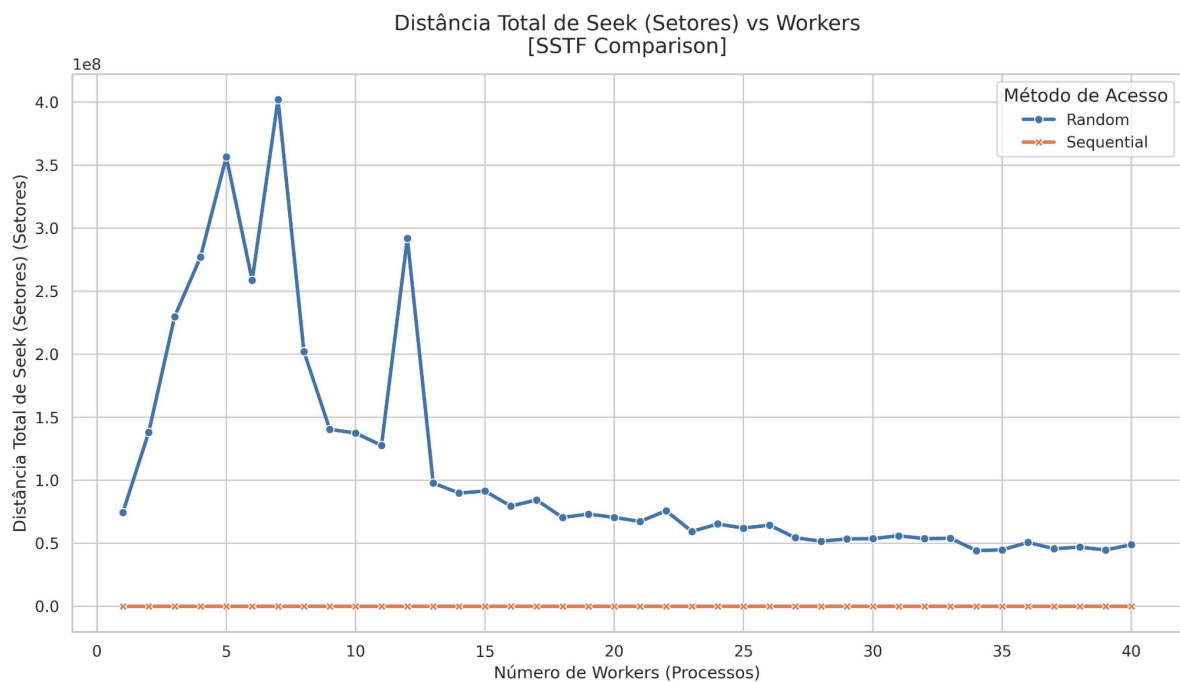


Latência Média: Tempo de Resposta vs Workers

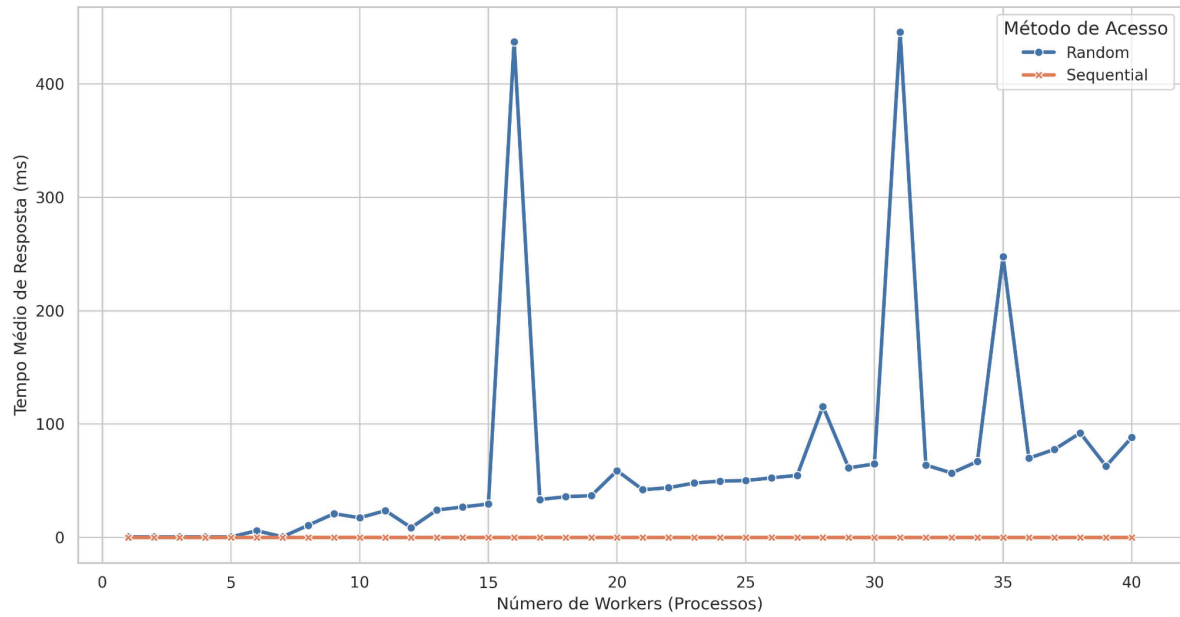




8. Gráficos SSTF



Tempo Médio de Resposta vs Workers
[SSTF Comparison]



Tempo Máximo de Resposta (Starvation) vs Workers
[SSTF Comparison]

